

Docker 部署指南（YOLO Web Builder v4）

本文件專門說明這個專案的 **Docker 化部署方式**，包含：

1. 在本機用 **Dockerfile** 建置映像檔
2. 將映像檔推到 Docker Hub
3. 在另一台機器安裝 Docker 並從 Docker Hub pull
4. 啟動容器前的先決條件
5. 使用 **docker run** 或 **docker compose** 啟動服務
6. 哪些套件／檔案會放進 image，哪些不會

這份 **guide.md** 的寫法，刻意沿用你上傳 README 的 Docker 教學脈絡與章節風格，但內容已改成符合 **這個專案本身** 的實際部署邏輯。

1. 先講清楚：這個 Docker image 的角色

這個專案的 Docker image，主要是拿來封裝 **Web Builder 服務本體**，也就是：

- FastAPI 後端
- 靜態前端頁面
- **pipeline_templates/** 樣板
- **backend/jobs/** 專案保存與 bundle 輸出位置

它的主要任務是：

- 提供瀏覽器 UI
- 接收表單設定
- 產生 bundle ZIP
- 保存 project JSON
- 必要時透過 SSH 觸發遠端執行

這個 image **不負責** 的事情

這個 image **不是** 用來把完整的 YOLO 訓練／匯出環境全部包進來。也就是說，它不會預先塞入：

- Ultralytics 的完整訓練環境
- TensorRT 執行期
- TFLite / TensorFlow 的整套轉檔環境
- 目標機器的 GPU driver

原因很簡單：

1. 真正的訓練／匯出通常是在 **另一台目標機** 上進行。
2. 這些依賴很重，而且跟目標機的 CUDA / TensorRT / CPU 架構密切相關。
3. 你這個專案原本就已經把 bundle 依賴拆到 **requirements_pipeline.txt** 中，較符合實務部署方式。

因此，這份 Dockerfile 的正確定位是：

- 把 Web Builder 自己封裝好，讓它能穩定部署在不同主機上
 - 把真正的訓練／匯出依賴留給生成出來的 bundle 在目標主機安裝
-

2. 這份 Dockerfile 會事先準備什麼

新的 `Dockerfile` 會預先處理以下內容：

2.1 Python 執行環境

- `python:3.11-slim` 作為基底
- 啟用：
 - `PYTHONDONTWRITEBYTECODE=1`
 - `PYTHONUNBUFFERED=1`
 - `PIP_NO_CACHE_DIR=1`

2.2 系統套件

- `bash`
- `ca-certificates`
- `curl`
- `openssh-client`
- `tini`

這些套件的用途大致如下：

- `curl`：健康檢查與基本連線測試
- `openssh-client`：容器內需要額外測 SSH 連線時可直接用
- `tini`：當 PID 1，較穩定地處理 signal 與子行程

2.3 Python 套件

依 `backend/requirements.txt` 安裝：

- `fastapi`
- `uvicorn`
- `jinja2`
- `pydantic`
- `python-multipart`
- `pyyaml`
- `paramiko`

2.4 會被 copy 進 image 的專案內容

- `backend/`
- `pipeline_templates/`
- `docker-compose.yml`
- `.env.example`
- `start.sh`
- `README.md`

2.5 持久化資料位置

- 容器內的保存位置是：`/app/backend/jobs`
- Dockerfile 也宣告了：

```
VOLUME ["/app/backend/jobs"]
```

但實務上，還是建議你在部署時使用 **bind mount**，讓主機端能直接看見 `jobs/` 內容。

3. 為什麼這樣切分最合理

你目前的專案程式結構其實已經把責任分得很清楚：

- Web 服務：`backend/`
- 生成出來的訓練／匯出模板：`pipeline_templates/`
- 使用者保存與 bundle 輸出：`backend/jobs/`

也就是說，容器本身只要能穩定執行 Web 服務即可。

而 bundle 的實際訓練依賴，已經在程式中按需求寫進 `requirements_pipeline.txt`；例如只有勾選 TFLite 匯出時，才加入對應 TensorFlow/TFLite 套件。這種設計比把所有重依賴都塞進 Web 容器更合理，也比較容易跨機部署。

4. 主機端的先決條件

要部署這個 Web Builder 容器，主機端通常只需要下列條件：

必要條件

- 64-bit Linux 主機（Ubuntu 最常見）
- Docker Engine
- 網路可連到 Docker Hub
- 能夠開放你要的 HTTP port（預設 8000）

若要使用 `docker compose`

還需要：

- Docker Compose plugin

若只是部署 Web Builder 本身

不需要 NVIDIA GPU，也不需要 NVIDIA Container Toolkit。

這點很重要：

這個容器本身只是 Web 服務，不是訓練容器；除非你刻意要把「生成 bundle 後，還在同一台主機自己跑訓練」也一起做，否則部署 Web Builder 的那台機器可以是一般 CPU 主機。

若你打算在同一台機器上額外承接訓練／匯出

那麼訓練用的目標主機才需要另外準備：

- 正確的 GPU driver
- 對應 CUDA / TensorRT / 其他推論環境
- 或者該主機自己的 Python / venv / conda 環境

這部分與 Web Builder 容器本身是分開的。

5. 在本機建置 Docker image

進入專案根目錄後執行：

```
docker build -t yolo-web-builder:v4 .
```

如果你想同時帶上 Docker Hub 命名，可直接這樣建：

```
docker build -t <YOUR_DOCKERHUB_USERNAME>/yolo-web-builder:v4 .
```

若你希望建置時優先拉最新 base image，可用：

```
docker build --pull -t <YOUR_DOCKERHUB_USERNAME>/yolo-web-builder:v4 .
```

Docker 的 **build** 會用 Dockerfile 與 build context 來建立 image；而 Compose 也可以依 Compose 檔中的 **build** 設定來重建服務 image。官方文件對這些行為有明確說明。

[cite](#)[turn111460search13](#)[turn915475search11](#)[turn915475search8](#)

建置完成後先檢查

```
docker images | grep yolo-web-builder
```

本機快速啟動測試

```
docker run --rm -it -p 8000:8000 yolo-web-builder:v4
```

然後開啟：

```
http://localhost:8000
```

6. 本機使用 bind mount 啟動（推薦）

因為這個專案需要把使用者保存的專案 JSON、bundle、ZIP 等資料保留下來，所以建議把 `backend/jobs/` 掛出來。

先在主機建立資料夾：

```
mkdir -p backend/jobs/projects
```

再執行：

```
docker run -d \
  --name yolo-web-builder \
  -p 8000:8000 \
  -e JOBS_DIR=/app/backend/jobs \
  -e TRUST_PROXY_HEADERS=1 \
  -v "$(pwd)/backend/jobs:/app/backend/jobs" \
  --restart unless-stopped \
  yolo-web-builder:v4
```

使用 bind mount 後，主機目錄會直接掛進容器；這正是 Docker 官方說明的 bind mount 用途：讓主機與容器共享同一份檔案路徑。 [\[cite\]turn915475search1\[turn915475search13\]](#)

檢查狀態

```
docker ps
docker logs -f yolo-web-builder
```

停止與刪除

```
docker stop yolo-web-builder
docker rm yolo-web-builder
```

7. 使用既有 `docker-compose.yml` 啟動

你這個專案本身就已經有 `docker-compose.yml`、`.env.example` 與 `start.sh`，所以如果你是以完整 repo 部署到主機，直接走 Compose 會最順。專案現有 Compose 做法本來就已經把 `./backend/jobs` bind mount 到容器內，並可透過 `APP_UID` / `APP_GID` 減少 Linux 權限不一致造成的寫入問題。

這種寫法也符合 Docker Compose 的角色：用來定義並執行多容器或單服務應用，並透過 `docker compose up` 進行 build、建立與啟動。 [\[cite\]turn915475search5\[turn915475search2\]](#)

步驟 1：建立 `.env`

```
cp .env.example .env
```

必要時把 `.env` 內的值改成：

```
PORT=8000
APP_UID=1000
APP_GID=1000
TRUST_PROXY_HEADERS=1
```

若你主機帳號的 UID / GID 不是 1000，可用：

```
id -u
id -g
```

把查到的數值寫進 `.env`。

步驟 2：啟動 Compose

```
docker compose up -d --build
```

步驟 3：檢查狀態

```
docker compose ps
docker compose logs -f web
```

步驟 4：停止

```
docker compose down
```

8. 上傳到 Docker Hub

8.1 登入 Docker Hub

```
docker login
```

Docker 官方目前說明：對 Docker Hub 而言，`docker login` 預設可以使用 device code flow；如果有指定 `--username`，則會改用輸入帳號的登入方式。 [\[cite\]turn111460search2](#)

若你想明確指定帳號：

```
docker login --username <YOUR_DOCKERHUB_USERNAME>
```

若你有啟用 2FA 或做自動化流程，建議改用 Docker Hub PAT，而不是密碼。PAT 是 Docker 官方建議的較安全替代方式。 [\[cite\]turn111460search14](#)

8.2 tag image

```
docker tag yolo-web-builder:v4 <YOUR_DOCKERHUB_USERNAME>/yolo-web-builder:v4
```

8.3 push image

```
docker push <YOUR_DOCKERHUB_USERNAME>/yolo-web-builder:v4
```

Docker Hub 官方文件也明確要求：推送前要先正確 tag 成 `namespace/repository:tag` 形式，再執行 push。 [\[cite\]turn111460search11](#) [\[turn111460search3\]](#) [\[turn111460search21\]](#)

9. 另一台機器要先裝什麼

如果你要在另一台 Ubuntu 機器上直接 pull 這個 image，通常建議先安裝：

- Docker Engine
- Docker Compose plugin（若你要用 Compose）

Docker 官方建議在 Ubuntu 上透過 Docker 官方 apt repository 安裝 Docker Engine，並先移除可能衝突的舊版套件。支援的 Ubuntu 版本與安裝流程可直接依官方文件操作。 [\[cite\]turn111460search0](#)

Ubuntu 安裝 Docker（官方 repo 路線）

```
sudo apt remove $(dpkg --get-selections docker.io docker-compose docker-compose-v2 docker-doc podman-docker containerd runc | cut -f1)

sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
sudo tee /etc/apt/sources.list.d/docker.sources > /dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo
"${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

驗證安裝

```
sudo docker run hello-world
```

若要用非 root 執行 Docker

Docker 官方有提供 Linux post-install 步驟，可把使用者加入 `docker` 群組，之後不必每次都加 `sudo`。

`□cite□turn111460search1□`

```
sudo usermod -aG docker $USER
newgrp docker
```

10. 在另一台機器 pull 並啟動

10.1 pull image

```
docker pull <YOUR_DOCKERHUB_USERNAME>/yolo-web-builder:v4
```

10.2 準備持久化資料夾

如果你只打算靠 image 本身啟動，不使用完整 repo，可先自己建立一個 host path：

```
mkdir -p ~/yolo-web-builder-data/jobs/projects
```

10.3 用 `docker run` 啟動

```
docker run -d \
  --name yolo-web-builder \
```



```
-p 8000:8000 \  
-e JOBS_DIR=/app/backend/jobs \  
-e TRUST_PROXY_HEADERS=1 \  
-v "$HOME/yolo-web-builder-data/jobs:/app/backend/jobs" \  
--restart unless-stopped \  
<YOUR_DOCKERHUB_USERNAME>/yolo-web-builder:v4
```

10.4 檢查

```
docker ps  
docker logs -f yolo-web-builder
```

11. 如果是用完整 repo 在另一台機器部署

若你在另一台機器上不只是 pull image，而是也把這整份專案 repo 帶過去，那麼可以直接：

```
cp .env.example .env  
docker compose up -d --build
```

這種模式的好處：

- 直接沿用 repo 內現成的 `docker-compose.yml`
- `backend/jobs/` 保存位置清楚
- 維護者比較容易看到 `.env`、`start.sh`、`README`

12. 這台遠端機器需不需要 GPU 套件？

情境 A：只部署 Web Builder

不用。

你只需要：

- Docker Engine
- 網路
- 儲存空間
- 可開放的 8000 port（或你自訂的 port）

情境 B：同一台機器還要跑生成出來的 bundle

這時才需要看該訓練主機的環境需求。

若該主機要讓 Docker 容器使用 NVIDIA GPU，NVIDIA 官方目前建議安裝 `nvidia-container-toolkit`，再用 `nvidia-ctk runtime configure --runtime=docker` 設定 Docker runtime。

□cite□turn915475search0□turn915475search3□

但再次強調：

- 這不是 Web Builder 本身啟動的必要條件
- 這是「你另外要在同機器上跑 GPU workload」時才需要

13. 推薦的實際工作流

方案 1：最簡單、最穩

- 本機有完整 repo
- 用本機 `docker build` 建 image
- push 到 Docker Hub
- 另一台機器只 pull image
- 另一台機器只負責跑 Web Builder
- 訓練交給第三台 GPU 主機或 SSH 目標機

這是最符合你這個專案設計的方式。

方案 2：另一台機器也保留完整 repo

- 把 repo 一起帶過去
- `cp .env.example .env`
- `docker compose up -d --build`
- 直接沿用目前專案的 compose 結構

這適合之後還要持續維護這個 Web Builder 本身。

14. 建議你一併注意的事項

14.1 `backend/jobs/` 必須持久化

這是最重要的一點。

因為這個目錄包含：

- Save 的 project JSON
- 產生出的 bundle
- 部分執行結果

若不掛出來，容器刪掉後這些內容就一起消失。

14.2 `APP_UID` / `APP_GID` 是給 Compose 模式的重要補丁

若你使用 repo 內既有的 Compose，Linux 主機上請注意 `.env` 裡的 UID / GID 是否對應你目前登入帳號，否則常見現象就是：

- 介面顯示 Save 成功
- 但 host 端的 `backend/jobs/` 內容沒有正常寫入

14.3 不要把訓練重依賴硬塞進 Web 容器

這會讓 image：

- 體積更大
- build 更慢
- 更容易碰到 CUDA / TensorFlow / TensorRT 相依衝突
- 對「不同環境直接部署 Web Builder」反而更不友善

15. 參考資料

- Docker Ubuntu 安裝：<https://docs.docker.com/engine/install/ubuntu/>
- Docker Linux post-install：<https://docs.docker.com/engine/install/linux-postinstall/>
- Docker login：<https://docs.docker.com/reference/cli/docker/login/>
- Docker push：<https://docs.docker.com/reference/cli/docker/image/push/>
- Docker Hub push guide：<https://docs.docker.com/docker-hub/repos/manage/hub-images/push/>
- Docker bind mounts：<https://docs.docker.com/engine/storage/bind-mounts/>
- Docker Compose：<https://docs.docker.com/compose/>
- docker compose up：<https://docs.docker.com/reference/cli/docker/compose/up/>
- NVIDIA Container Toolkit：<https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>

另外，這份部署指南的章節安排，也參考了你上傳的 MM-GD Docker README 的教學脈絡與鋪陳方式。

□filecite□turn2file0□

16. 總結

對這個專案而言，最合理的 Docker 方案不是做一個「什麼都包」的巨大訓練 image，而是做一個：

- 可直接部署 Web UI
- 可持久化 jobs/
- 可推到 Docker Hub
- 可在另一台主機直接 pull/run
- 並保留對 SSH 遠端訓練的支援

新的 `Dockerfile` 與這份 `guide.md` 就是依這個思路撰寫的。